Khulna University of Engineering & Technology (KUET)
Department of Computer Science and Engineering

CSE 4102 · Computer Graphics and Image Processing Laboratory
4th Year, 1st Term

The Ramen Shop

A Cozy Japanese Ramen Restaurant in OpenGL



The shop at night, rendered in real time by the project.

Submitted by
Rajorshi Das
Roll: 2107060
Department of Computer Science and Engineering, KUET

Submitted to
Md Tajmilur Rahman
Lecturer
Department of Computer Science and Engineering, KUET

Md. Mubtashim Abrar Nihal
Lecturer
Department of Computer Science and Engineering, KUET

Built with C++17 · OpenGL (fixed function + GLSL 1.20) · FreeGLUT · GLEW
October 2026

Contents

1 Introduction **1**
- 1.1 Objectives 1
- 1.2 Tools 1
- 1.3 Features implemented 1

2 The Shop and Its Surroundings **2**

3 Methodology **2**
- 3.1 Implementation overview 2
- 3.2 Modelling from basic shapes 3
- 3.3 3D transformations 3
- 3.4 Textures and texture mapping 4
- 3.5 Illumination: ambient, diffuse and specular light 5
- 3.6 Materials, sky, day and night 5
- 3.7 Gouraud and Phong shading 6
- 3.8 Shadows 6
- 3.9 Glass refraction (screen space) 6
- 3.10 Ray tracing 7
- 3.11 Animation and interaction 8
- 3.12 The ramen cooking game 9

4 Results **9**
- 4.1 Outdoors 9
- 4.2 Ground floor 10
- 4.3 Upstairs tatami room and details 10
- 4.4 Summary and performance 11

5 Discussion **11**

6 Conclusion **12**

List of Figures

1 Floor plans of the two storeys 2
2 The rendering pipeline: work done once at start-up (left) and in every frame (right). 3
3 The construction primitives: every object in the shop is some combination of these. 3
4 Parent–child transformations 4
5 Procedural textures as they appear in the shop (crops of real screenshots). 4
6 Illumination terms and Gouraud shading 5
7 The light tour 5
8 Shadow methods 7
9 Glass refraction 7
10 Ray tracing steps 8
11 Ray tracing 8
12 Moving things in the street 8
13 The game: its state machine and three screens (HUD drawn with an orthographic projection). 10
14 The shop from outside and the lake behind it. 10
15 The ground floor. 10
16 The upstairs room, the porch details and the wind chime. 11

List of Tables

1 Tools and size of the project. 1
2 Features of the project and where each one is explained. 1
3 What is in each zone of the scene. 2
4 Modules of the program. 2
5 Where each 3D transformation is used. 4
6 The eight lights. At night all fixtures glow in their own warm colour; they flicker slowly. 5
7 The moving objects of the scene. 8
8 The camera modes 9
9 The keyboard and mouse controls. 9
10 Scene statistics 11
11 Parameters that can be changed to see their effect. 11

1 Introduction

A small ramen-ya sits on a quiet Japanese street. Its two-storey wooden shop has a dining counter with an open kitchen, two small tables, a cosy tatami room upstairs, a street with a sidewalk, cherry and maple trees, and a lake with ducks behind it. This project builds that whole place as an interactive, real-time **OpenGL** program. You can fly around it with seven different cameras, switch every lamp on and off one by one, change between day and night, click the doors open, and play a small ramen-cooking game.

Everything is made in code. Every object is built from a few basic shapes (cube, cylinder/cone, sphere, torus, a few lathed and swept surfaces), and every texture is painted by the program itself at start-up, so there are no model or image files to load. Lighting is done with the ambient, diffuse and specular terms of the Phong model, with per-vertex (Gouraud) and per-pixel (Phong) shading. On top of that there are shadows, a refractive glass, a small CPU ray tracer, and many moving things: a cyclist, a person, a cat, a rippling banner, a glass wind chime that swings as a real pendulum, and fans.

1.1 Objectives

1. Model a believable multi-part scene (shop, upstairs room, street, nature) only from basic shapes and 3D transformations.
2. Light it with the ambient, diffuse and specular terms of the illumination model, with several point, spot and directional lights that can each be switched on and off.
3. Apply procedurally generated textures with correct mapping, and shade with both Gouraud and Phong shading.
4. Add advanced effects: shadows (projected and shadow-mapped), screen-space refraction, and real ray tracing.
5. Make the scene alive and explorable: animation, a free and several fixed cameras, mouse and keyboard interaction, and a small game.

1.2 Tools

Language and build	C++17, Visual Studio (MSBuild), x64 Debug and Release
Graphics API	OpenGL fixed-function pipeline plus GLSL 1.20 shaders (Phong, glass refraction, ray-trace compositing); framebuffer objects and float textures (OpenGL 3.0)
Libraries	FreeGLUT (window, input), GLEW (extensions), GLU (quadrics, matrices); NuGet package `nupengl.core`
Size	about 15,000 lines of our own code in 18 source files and 17 headers; no model, image or sound files

Table 1: Tools and size of the project.

1.3 Features implemented

Feature	Where it is shown	Section
Modelling from basic shapes (cube, cylinder, cone, sphere, torus, lathe, grid)	shop, furniture, trees, props	3.2
3D transformations: translation, rotation, scaling, pivot rotation, hierarchies	cyclist, person, cat, doors, trees	3.3
Procedural textures, mipmaps, anisotropic filtering	wood, tile, plaster, tatami, grass, water	3.4
Ambient, diffuse, specular light; point, spot and directional lights; attenuation; sky, day and night (§3.6)	8 lights, keys `1-7`, `-`, `=`, `\`	3.5
Gouraud and Phong shading	key `G`	3.7
Shadows: projected (stencil) and soft shadow mapping with PCF	sun, lamps, key `Y`	3.8
Glass refraction (screen space)	tumbler on a table, key `Z`	3.9
Ray tracing (CPU): reflection, refraction, BVH	pots, bowls, glass, mirror, key `R`	3.10
Animation, perspective cameras, mouse and keyboard interaction	everywhere	3.11
Ramen cooking game (state machine, HUD with orthographic projection)	key `Enter`	3.12

Table 2: Features of the project and where each one is explained.

2 The Shop and Its Surroundings

One world coordinate system is used: x runs to the right, y points up and z points toward the street (the front of the shop). One unit is about one metre. The ground floor is 9.8×7.8 units ($x \in [-4.9, 4.9]$, $z \in [-3.9, 3.9]$) and 3.3 high; the upper floor adds 2.0, and the gable roof rises another 2.2 above that. Behind the shop the ground is a large lawn (the ground plane is 240 units wide) with a lake at $(0, -24)$; in front of it the road and two sidewalks run along x at $z \approx 12$, $z \approx 7$ and $z \approx 17$. Figure 1 shows the floor plans of the two storeys.



Figure 1: Floor plans (the street side of the shop is at the bottom). The tatami room is covered by 8 rows of mats in a staggered pattern.

Zone	Contents
Ground floor	Dining counter with 6 stools and a lacquered top; four ramen bowls with steam; open kitchen (two steel stock pots, hood, shelves, fridge, sink); two customer tables with two chairs each and a table lamp over each; paper box lanterns, hanging lanterns, a floor lantern tower, pendant glass lamps, an area-light panel; noren curtain, wall clock, shoji doors and windows
Upper floor	Tatami floor with cloth borders, plaster walls with timber frame, fusuma doors, board ceiling; futon bed, chabudai tea table and tea set, cushions, tansu chest, sideboard with radio, mirror, six wall pictures, windows, ceiling fan and stand fan
Outdoors	Street and two flagstone sidewalks, street lamps, cherry, maple, pine and broadleaf trees, bamboo, bushes, flowers, a neighbouring house, stone lanterns, lake with bridge, torii gate and two ducks

Table 3: What is in each zone of the scene.

3 Methodology

3.1 Implementation overview

The program is split into small modules (Table 4). `main.cpp` owns the window and the frame loop, `scene.cpp` and `exterior.cpp` draw the world, `shapes.cpp` holds the primitives and materials, and the effects each have their own file. At start-up the program builds everything it needs once; every frame it then reads input, updates the animation, and draws the world (Figure 2).

Files	Role
`main`, `camera`	window, frame loop, timers, the seven cameras, keyboard and mouse input
`shapes`, `texture`	primitives, materials (gloss, emission), procedural textures
`scene`, `exterior`	ground, sky, shop, upstairs room, trees, props, wind chime, shadows
`furniture`, `food`, `decorations`	counter, kitchen, tables, bowls, lanterns, noren, clock
`lighting`, `shader`	the 8 lights, day/night, Phong shader, fog, interior light scope
`shadowmap`, `refraction`	soft shadow mapping, screen-space glass refraction
`rtcore`, `rtscene`, `rtpass`	CPU ray tracer, the shop as the rays see it, threaded pass and compositing
`objects`, `game`	selectable and movable objects, the ramen cooking game

Table 4: Modules of the program.

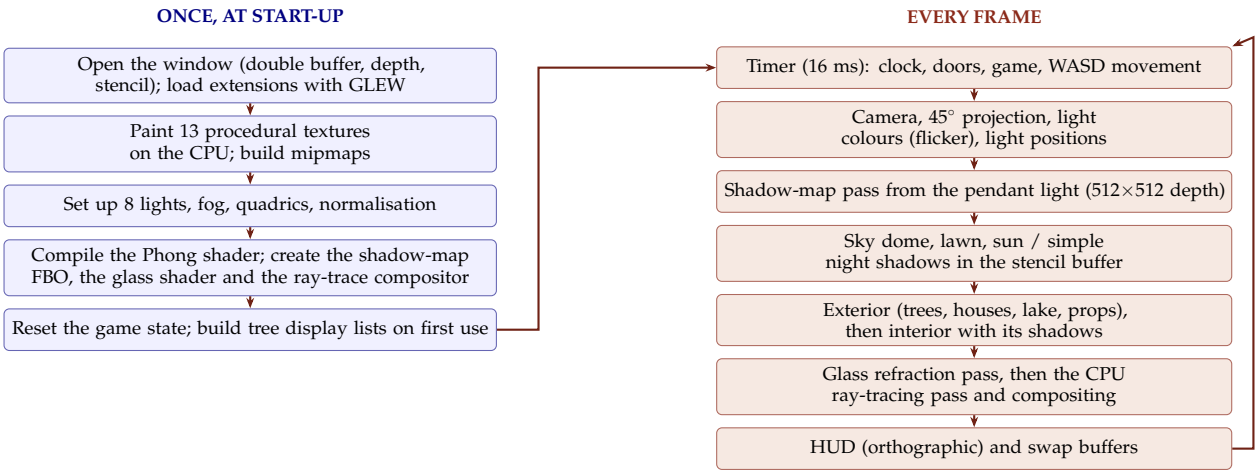


Figure 2: The rendering pipeline: work done once at start-up (left) and in every frame (right).

3.2 Modelling from basic shapes

Every object is assembled from a handful of primitives in `shapes.cpp`: a cube (`glutSolidCube`), a cylinder and cone made by one routine with a bottom and a top radius (a frustum when both are different), a sphere (`gluSphere`), a torus, and planes and wedges. Boxes of other sizes are cubes scaled by non-uniform factors, ellipsoids are scaled spheres, and a tube between two points is a cylinder rotated to point from one end to the other. Three more constructions are used for special shapes:

- **Lathe (surface of revolution).** A profile of (r_i, y_i) pairs is swept about the y axis: $(r_i \cos\theta,\ y_i,\ r_i \sin\theta)$ with outward normal proportional to $(\Delta y \cos\theta,\ -\Delta r,\ \Delta y \sin\theta)$ from the profile tangent. The bell-shaped table lamps, the wind-chime bell and the paper lanterns are made this way.
- **Grid surface.** The nobori banner is a 12×28 grid of quads whose depth is $z(u, v, t) = \big(0.09 \sin(3.2t - 4u + 1.5v) + 0.015 \sin(5.1t + 3v)\big)u$ (see §3.11); the normal comes from the slope $\partial z/\partial u$, so the folds are shaded correctly.
- **Recursive trees.** A branch is two tapered tubes; each branch spawns 3 (first level) or 2 (later levels) child branches at random angles $\theta \in [0.45, 0.90]$ rad, and the tips carry leaf or blossom clusters. The result is compiled into OpenGL display lists once, and only the swaying transform is applied per frame.

Details such as the ramen toppings, the cat, the cyclist and the pictures on the wall are all made from the same primitives: for example every wall picture is drawn from rectangles, ellipses, triangles and thick lines, and shapes that would spill out of a frame are cut with the Sutherland–Hodgman clipping algorithm.

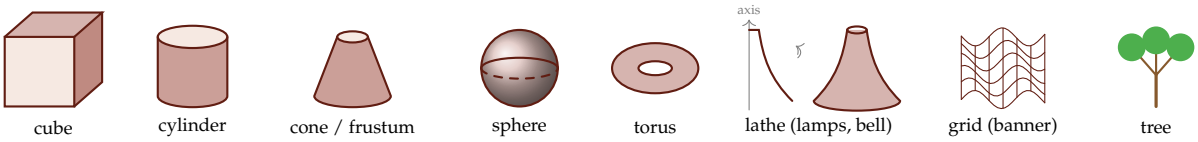


Figure 3: The construction primitives: every object in the shop is some combination of these.

3.3 3D transformations

All placement goes through one function that applies the standard composite transform to the modelview matrix:

$$M = T(\mathbf{p})\, R_x(\alpha)\, R_y(\beta)\, R_z(\gamma)\, S(s_x, s_y, s_z), \tag{1}$$

so a part is scaled first, then rotated, then moved. Nesting objects with `glPushMatrix/glPopMatrix` gives hierarchical models: a child inherits everything its parent does. Table 5 lists where each kind of transformation is used, and Figure 4 shows the joint hierarchies of the three articulated characters.

Transformation	Used for
Translation	cyclist $x(t) = x_0 + vt \bmod L$, walking person and cat, sliding shoji doors, rising steam, ducks on a circle $(3.5\cos 0.4t,\ 3.5\sin 0.4t)$
Rotation	wheels $\theta_w = -d/(rS)$ (rolling without slipping), pedals and cranks, fan blades, hinged entrance door, wind-chime pendulums, head of the stand fan swinging $\pm 35°$
Scaling	stretching cubes into planks and counters; cat $\times 1.8$, bicycle $\times 1.4$; user scaling of selected objects (keys , and .); breathing of the noodle steam
Rotation about a pivot	$T(\mathbf{c})R(\theta)T(-\mathbf{c})$ for swaying tree branch groups, noren curtains, hanging lanterns, the banner's top bar
Projection (shear)	flattening the scene onto the ground for shadows (§3.8)

Table 5: Where each 3D transformation is used.

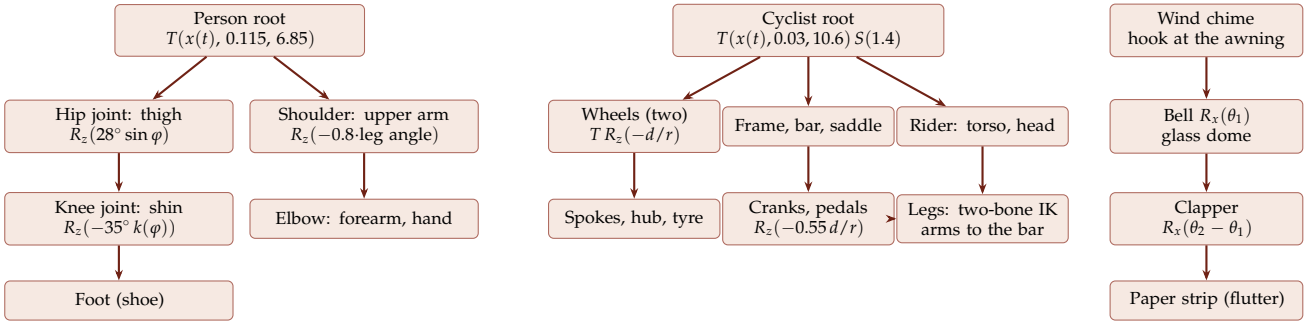


Figure 4: Parent–child transformations of the walking person, the cyclist and the wind chime. A joint moves everything below it; the dashed arrow means the rider's feet follow the pedals.

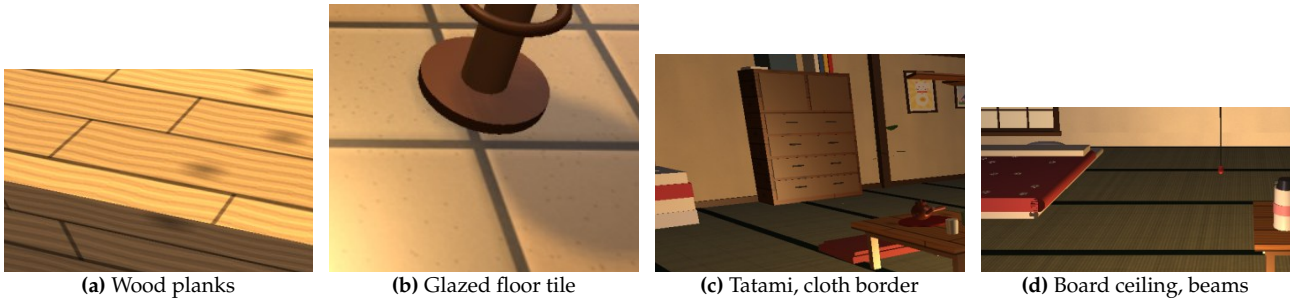
Two-bone legs. For the cyclist the knee position is found from the hip $\mathbf{h}$, the pedal $\mathbf{f}$ and the bone lengths L_1, L_2 with the law of cosines. With $d = |\mathbf{f} - \mathbf{h}|$ clamped to $L_1 + L_2$,

$$a = \frac{L_1^2 - L_2^2 + d^2}{2d}, \qquad h = \sqrt{L_1^2 - a^2}, \qquad \mathbf{k} = \mathbf{h} + a\,\hat{\mathbf{u}} + h\,\hat{\mathbf{u}}^{\perp}, \tag{2}$$

so the knee always bends forward and the shin follows the pedal exactly. The cat's step uses a simple swing: each paw moves $0.06\sin(\varphi + \varphi_i)$ with diagonal pairs in anti-phase and is lifted by $0.025\max(0, \cos(\varphi + \varphi_i))$.

3.4 Textures and texture mapping

There are 13 procedural textures, each painted in a function when the program starts: wood planks, dark wood, glazed floor tile, lime plaster, asphalt, concrete, roof tile, a lawn (with a large-scale mottling map that hides the tiling), a water ripple, a soft cloud sprite, the sphere-mapped environment of the room, and the woven tatami. Most use value noise summed over several octaves (fractional Brownian motion) built on an integer hash, made tileable by wrapping the lattice. For example the tatami reeds use a rounded profile across the rows, $k = 0.78 + 0.22\sin(\pi f)$ + row tone + fibre + mottling, where f is the fractional row coordinate. Textures use `GL_MODULATE` so the lighting still darkens and tints them, mipmaps with trilinear filtering, and anisotropic filtering where the GPU offers it. Boxes are mapped planarly: the texture coordinates of each face are its size divided by a scale, so one repeat always covers the same real-world length (a plank is always the same width whatever the box). Sphere-mapped texture coordinates (`GL_SPHERE_MAP`) give the pots and glass a faint reflection of the room. Figure 5 shows four of them in the scene.



(a) Wood planks **(b)** Glazed floor tile **(c)** Tatami, cloth border **(d)** Board ceiling, beams

Figure 5: Procedural textures as they appear in the shop (crops of real screenshots).

3.5 Illumination: ambient, diffuse and specular light

OpenGL gives 8 lights, and each light i has its own ambient, diffuse and specular colour. The colour of a surface point with unit normal $\mathbf{N}$, light direction $\mathbf{L}$ and half vector $\mathbf{H} = \widehat{\mathbf{L}+\mathbf{V}}$ is

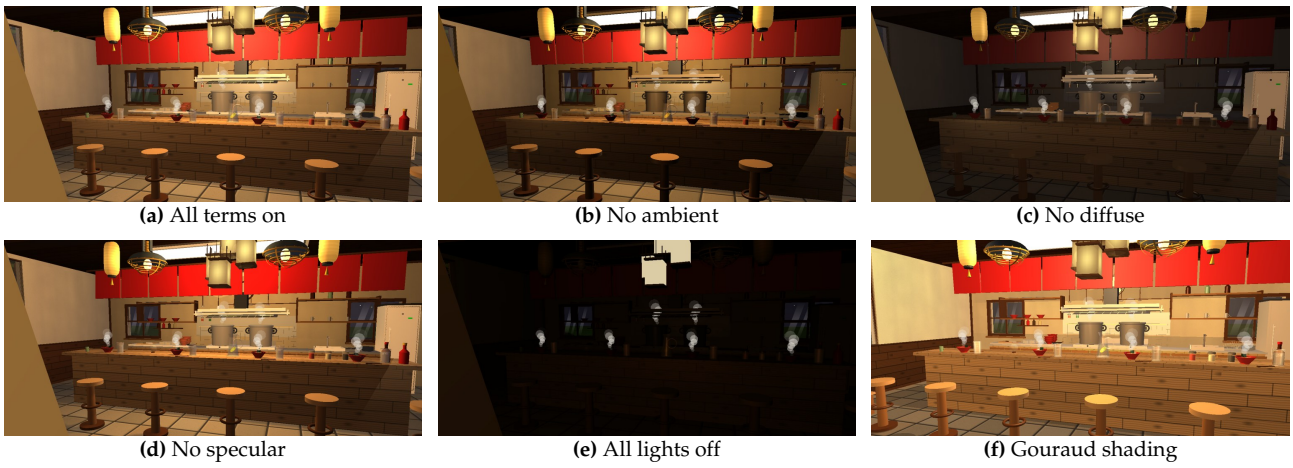
$$I = \mathbf{e} + \sum_i a_i[\mathbf{c}_a\mathbf{l}_{a,i} + \mathbf{c}_d\mathbf{l}_{d,i}\max(\mathbf{N}\cdot\mathbf{L}_i,0) + \mathbf{c}_s\mathbf{l}_{s,i}\max(\mathbf{N}\cdot\mathbf{H}_i,0)^n], \qquad a_i = \frac{\sigma_i}{k_c + k_l d + k_q d^2}, \tag{3}$$

where $\mathbf{e}$ is the material's emission (glowing lantern paper, bulbs), $\mathbf{c}_{a,d,s}$ are the material colours (the specular colour and the shininess n are set per material: matte plaster $n=8$, lacquered wood $n\approx55$, glass $n=140$), and a_i is the attenuation. For a spot light $\sigma_i = (\cos\alpha)^e$ inside the cone ($\alpha \le$ cut-off, here 28°, $e = 35$) and 0 outside. The global ambient light is zero, so a lamp that is off really lights nothing.

Table 6 lists the lights. Only 8 exist, but the shop needs more, so the four lights that normally serve the outside (entrance lanterns, street lamps, sign) are re-assigned to interior roles while the inside is drawn, and put back afterwards; the shader only sees the interior set for interior objects. Keys `1`–`7` switch each fixture on its own, `0` the sun/moon, and `-`, `=`, `\` remove just the ambient, diffuse or specular term from every light at once. Figure 6 shows the effect of each term.

Light	Type	Position	Role (outside / inside the shop)
0	directional	$(0.45, 0.70, -0.45)$	sun (warm white) or moon (blue-silver)
1	point, $k_{l,q} = 0.14, 0.07$	$(0, 2.85, 0)$	dining pendant lamps (key `1`)
2	point	$(-3.8, 2.5, 4.5)$ / $(0, 2.6, 4.7)$	left entrance lantern / light leaking through the paper wall
3	point	$(3.8, 2.5, 4.5)$ / merged	right lantern / box lanterns, hanging lanterns and ceiling panel (keys `3`–`5`)
4	spot 28°, $e = 35$	$(-2.4, 3.25, 2.8)$	left table lamp (key `2`)
5	point	$(0, 3.4, 7)$ / $(2.4, 3.25, 2.8)$	street lamps / right table lamp, spot (key `2`)
6	point	$(0, 4.5, 4.2)$ / $(-4.1, 1.4, 3.2)$	shop-sign glow / floor lantern tower (key `6`)
7	point	$(0, 5.15, 0)$	upstairs paper lamp (key `7`)

Table 6: The eight lights. At night all fixtures glow in their own warm colour; they flicker slowly.



(a) All terms on **(b)** No ambient **(c)** No diffuse

(d) No specular **(e)** All lights off **(f)** Gouraud shading

Figure 6: (a)–(e) The illumination terms switched off one at a time, at night. Without ambient the shadows are slightly deeper; without diffuse only the glow and highlights remain; without specular the shiny counter and pots go flat; with every light off only self-lit surfaces remain. (f) The same view with per-vertex Gouraud shading.



(a) Pendant lamps (`1`) **(b)** Table lamp and chairs (`2`) **(c)** Box lanterns (`4`) **(d)** Lantern tower (`6`)

Figure 7: The light tour (key `8`) flies the camera to each fixture and switches it on, so every lamp can be seen with its own glow and light pool.

3.6 Materials, sky, day and night

Materials. Besides colour, each surface has an emission, a specular colour and a shininess. For ordinary dielectrics (wood, ceramic, cloth) the specular colour is white and its strength and sharpness follow the

roughness $r \in [0, 1]$:

$$n = 4 + 124(1 - r)^2, \qquad \rho_s = 0.08 + 0.55(1 - r), \tag{4}$$

so lacquer ($r \approx 0.1$, $n \approx 105$) has a tight, bright highlight and cloth ($r \approx 0.9$, $n \approx 5$) only a faint broad one. For metals the highlight is tinted by the metal colour, which is why gold knobs look gold and steel looks grey. Most surfaces get a small default sheen ($\rho_s = 0.14$, $n = 24$) so nothing looks flat.

Sky. The sky is a half sphere of 14 rings drawn first, with depth writes off and the camera translation removed from the modelview matrix, so it stays infinitely far away (the skybox technique) and the scene is drawn over it. The colour of ring k blends from the horizon colour to the zenith colour with $t = (e/90^\circ)^{0.55}$, where e is its elevation; by day the horizon equals the fog colour so that far ground melts into the sky. The sun and moon are camera-facing discs with additive halos; at night 132 stars and a cratered moon are added, and soft cloud sprites drift by day.

Day and night (key `T`). One switch changes the sky, the sun/moon light (warm gold 0.85, 0.78, 0.55 by day; cool blue 0.14, 0.16, 0.25 at night), the strength of every lantern and window glow (dimmed to about 15–20% by day), the street lamps, fog density (0.012 or 0.022), the fireflies and the sun shadows (day only).

3.7 Gouraud and Phong shading

With key `G` the whole scene switches between two shading models.

- **Gouraud shading** (fixed function): the lighting equation is evaluated at each vertex and the colour is interpolated across the polygon. It is fast, but a highlight that falls between vertices disappears, which is visible on the large lacquered counter and floor (Figure 6f).
- **Phong shading** (GLSL 1.20, `shader.cpp`): the normal is interpolated and the same equation is evaluated for every pixel, for all 8 lights, with spot cones, attenuation, a per-light enable mask and fog. The specular term uses the Blinn half vector, which gives the broad glossy streaks seen on lacquered wood and steel.

The vertex shader passes the eye-space position, normal, colour and texture coordinate; the fragment shader multiplies the interpolated colour with the texture, applies the lighting, then mixes in exponential-squared fog $f = e^{-(\rho z)^2}$. Unlit things (steam, text, outlines) go through the same program with lighting turned off.

3.8 Shadows

Three kinds of shadow are used (Figure 8).

Outdoor sun shadows. The whole outdoor scene is drawn a second time, flattened onto the ground along the sun direction $\mathbf{s} = (s_x, s_y, s_z)$ with a shear matrix that sends every point to

$$x' = x - \frac{s_x}{s_y}\, y, \qquad y' = h, \qquad z' = z - \frac{s_z}{s_y}\, y. \tag{5}$$

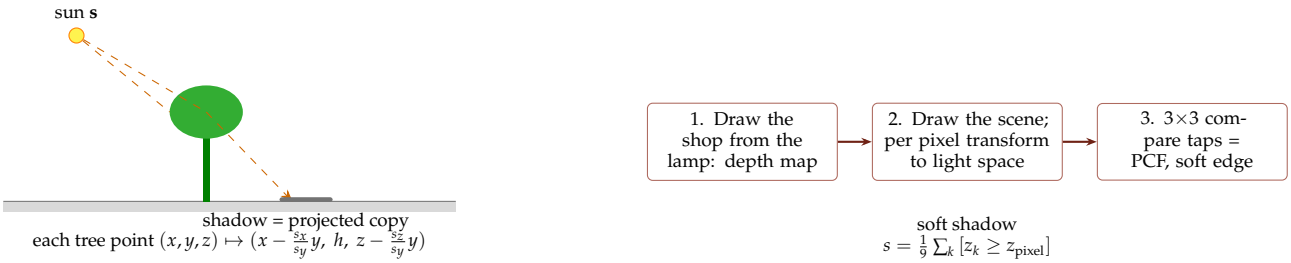
The flattened drawing goes only into the **stencil buffer** (colour and depth writes off, the stencil value is inverted for the first fragment that covers a pixel and the test then fails for later ones). One soft dark layer is then blended over the marked pixels, so overlapping branches never darken twice and every shadow has exactly the outline of its object. Trees near the shop use their real leaf geometry; trees far away use cheap solid crown shapes. Depth testing is turned off for this pass, because the shadow plane lies a few millimetres above the paving and would otherwise z-fight. At night only a few simple, fixed soft shadows (under the cherry tree and the posts by the shop) are drawn, because moving shadows from the street lamps looked noisy.

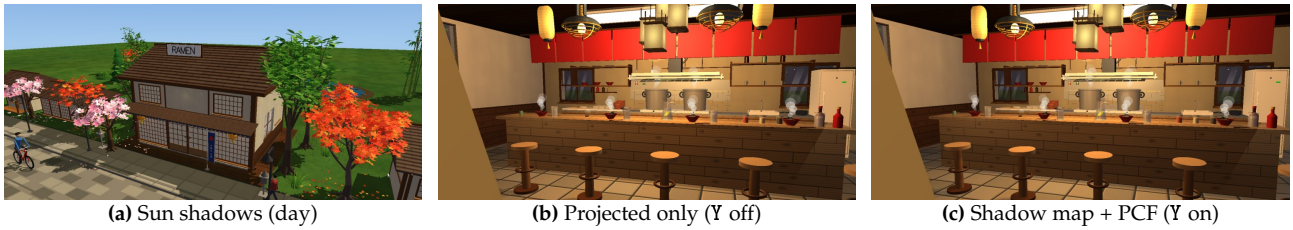
Interior shadows. The furniture is projected onto the tiled floor from the pendant and table lamps with the *point-light* version of the same matrix, $x' = \dfrac{l_y x - l_x y}{l_y - y}$, and a stencil counter ($1 \to 2 \to 1$) makes each layer darken once.

Soft shadows (key `Y`). The counter, stools, tables, chairs and bowls are drawn from the dining pendant light into a 512×512 depth texture (FOV 120°, polygon offset against acne). In the Phong shader every pixel is transformed into the light's clip space and compared to the map with hardware depth compare, using 3×3 taps spaced 1.5 texels apart, each already a bilinear 2×2 compare, a percentage-closer filter (PCF). The result is a smooth penumbra instead of a hard edge.

3.9 Glass refraction (screen space)

The empty tumbler on the left table is drawn last, over the finished picture. The screen area behind it is copied into a texture, and the tumbler is drawn with a shader that moves the lookup into that texture by the amount `refract()` bends the view ray at the glass normal (index 1/1.5), so the table and floor behind it appear shifted and distorted. A Fresnel reflection $F = 0.04 + 0.96(1 - \cos\theta)^5$ (Schlick's approximation) adds the shine of the

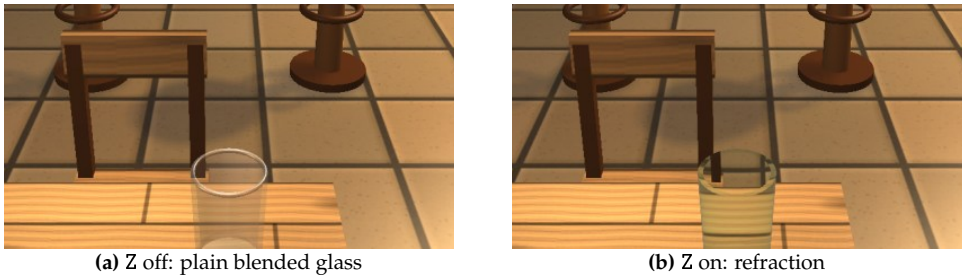




(a) Sun shadows (day) **(b)** Projected only (Y off) **(c)** Shadow map + PCF (Y on)

Figure 8: Top: how a tree is projected onto the ground, and the steps of soft shadow mapping. Bottom: (a) projected tree, bicycle and person shadows; (b), (c) the same view of the dining area without and with the shadow map, which adds soft shadows of the stools, bowls and counter.

warm room at grazing angles, and the lamps leave sharp highlights ($n = 140$). This is an approximation: only what is already on the screen can be seen through the glass (Figure 9).



(a) Z off: plain blended glass **(b)** Z on: refraction

Figure 9: The tumbler on the left table without and with screen-space refraction: the floor and table edge behind it are bent and the wall of the glass reflects the room.

3.10 Ray tracing

Key R adds a real ray tracer that runs on the CPU for a few featured objects: the two steel stock pots, the four ramen bowls, a glass ball, the drinking glasses and water jug, and a mirror in the upper room. The rest of the scene is still drawn by OpenGL; the ray tracer only decides the pixels of these objects (Figures 10 and 11).

- **Scene.** `rtscene.cpp` rebuilds the shop for the rays from the same positions and sizes as the OpenGL models, with boxes, truncated cones, disks and spheres, the same textures, and the live OpenGL lights. Lamps are light-emitting shapes, so they show in reflections.
- **Intersection.** Exact formulas: slab test for boxes, quadratic equations for spheres and cones, plane test for disks. A **bounding volume hierarchy** (BVH) over the small shapes (median split along the longest axis, at most 2 shapes per leaf) is traversed for every ray; the large floors and walls are tested directly.
- **Shading.** Surfaces reached are lit with the same Blinn–Phong model. Metals, glaze, soup and the mirror spawn a reflection ray weighted by Schlick's Fresnel term, up to three bounces. In glass and water every surface applies **Snell's law** $n_1 \sin\theta_1 = n_2 \sin\theta_2$ (glass $n = 1.5$, water $n = 1.33$), with the refracted direction

$$\mathbf{t} = \eta\,\mathbf{d} + \left(\eta\cos\theta_1 - \sqrt{1 - \eta^2(1 - \cos^2\theta_1)}\right)\mathbf{n}, \qquad \eta = n_1/n_2, \tag{6}$$

  and total internal reflection when the root is imaginary.
- **Speed.** Only the screen rectangles of the featured objects are traced, the rows are shared by all CPU threads (up to 8), and a ray budget adapts to keep the pass near 25 ms: when a large area is in view, one ray is traced per 2×2 or larger block of pixels.
- **Compositing.** The result is drawn over the OpenGL frame in a small shader only where the OpenGL depth buffer shows the same surface, so a nearer object still hides a pot or glass.

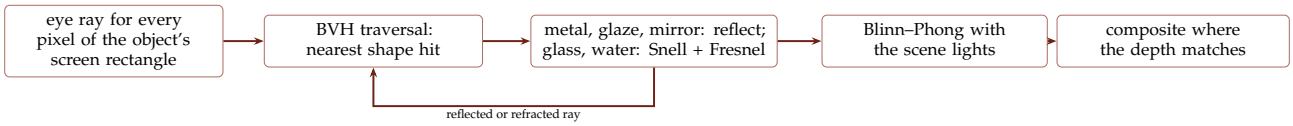
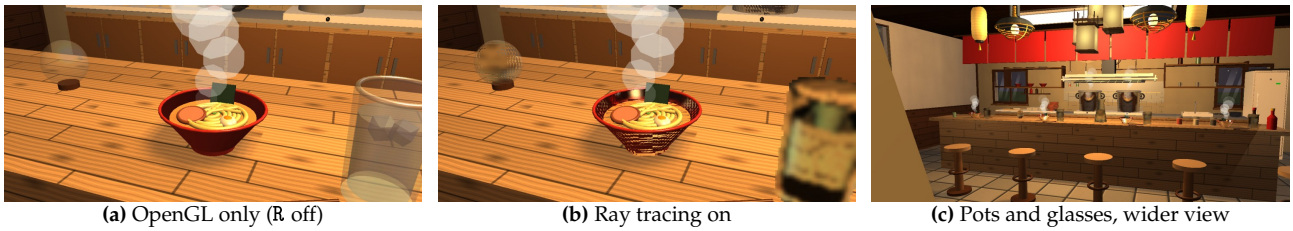


Figure 10: The steps taken for every eye ray of a ray-traced object.



(a) OpenGL only (R off) **(b)** Ray tracing on **(c)** Pots and glasses, wider view

Figure 11: (a), (b) Close view of a bowl without and with ray tracing: the glaze mirrors the light, the glass ball refracts the counter and the water glass bends what is behind it. (c) A wider view with the pots reflecting the room. At this distance one ray covers several pixels, so ray-traced objects look slightly blocky.

The cost depends on the build: measured in a close view with 6–7 featured objects on screen (about 7,000 eye rays) on 8 threads, the ray-tracing pass takes about **13 ms in a Release build** and about 80 ms in Debug, so Release is recommended with R on.

3.11 Animation and interaction

All motion runs from one clock, animTime, advanced by a 16 ms timer, so the space bar pauses everything at once.

Object	Motion
Cyclist	rides from $x = -16$ to 16 at 7 units/s and restarts; wheels turn by d/r (no slipping), cranks by $0.55\,d/r$, legs follow the pedals by two-bone IK; the headlight glows and throws a beam at night
Person	walks right to left at 1.8 units/s with 28° hip swing, knees that bend while the leg swings forward, arms swinging against the legs, and a $\lvert\cos\rvert$ body bob
Cat	walks left to right at 3 units/s; every third lap it stops in front of the shop, turns, sits and washes its face (paw lick, wipe over the ear, head tilt) before walking on
Nobori banner	grid of quads displaced by a travelling wave, calm at the pole and stronger at the free edge (the wave of §3.2)
Wind chime	two coupled pendulums with real physics (below)
Fans	ceiling fan blades at 160°/s; stand fan blades at 900°/s, head swinging $35°\sin(0.6t)$
Nature	wind sway of every tree branch group $A[\sin(1.1t+\phi)+0.45\sin(2.3t+1.7\phi)+0.25\sin(0.37t+0.5\phi)]$, falling petals, ducks on circles, fireflies, ripples
Doors	entrance door hinge 2.5°/frame, sliding doors 2.5%/frame, opened by a mouse click

Table 7: The moving objects of the scene.



Figure 12: Moving things in the street: the cyclist, the person and the cat (day view), and the glass wind chime under the awning of the shop.

Wind chime physics. The bell (angle θ_1, length $L_1 = 0.12$ m) and the clapper with its paper strip (θ_2, $L_2 = 0.20$ m) are pendulums driven by a gusty wind $w(t)$ and integrated with 8 semi-implicit Euler sub-steps per frame:

$$\ddot{\theta}_1 = -\tfrac{g}{L_1}\sin\theta_1 - c_1\dot{\theta}_1 + k_1 w(t), \qquad \ddot{\theta}_2 = -\tfrac{g}{L_2}\sin\theta_2 - c_2\dot{\theta}_2 + k_2 w(t). \tag{7}$$

The paper strip catches far more wind than the glass ($k_2 = 22$ against $k_1 = 3$), so the clapper swings further; when the relative angle $\theta_2 - \theta_1$ reaches 0.42 rad it hits the glass, bounces back with -0.45 of the relative velocity, and a star-shaped glint flashes on the rim and fades.

Interaction. A click is turned into a world ray with `gluUnProject` at the near and far plane, intersected with the plane of each door ($t = (z_0 - n_z)/d_z$), and the hit point is tested against the door's rectangle. Keys select an object (Tab) and move, rotate and scale it with the delta-transform system. The cameras are listed in Table 8; the controls are in the project's `README`.

Camera	Behaviour
Orbit (default)	orbits the shop; arrows, wheel and mouse drag change angle, distance and height
Walkthrough (`C`)	first person; `WASD` to move, mouse to look, stairs followed automatically, clamped in game mode
Counter, Upper Room, Pond (`C`)	orbit inside the counter area, the tatami room, and around the lake with the ducks
Bowl close-up (`B`)	orbits a ramen bowl at 0.7 m, a good place to see steam and glass
Light tour (`8`)	flies to the next lamp, switches it on; `9` toggles it

Table 8: The seven camera modes. Perspective projection (45°, 62° in game mode) is used for the world and an orthographic projection for the HUD.

Key	Action	Key	Action
`C, B, 8, 9`	next camera, bowl close-up, light tour, toggle tour lamp	`T`	day / night
Mouse drag, wheel	orbit or look; zoom or height	`0`	sun / moon on / off
Arrows, PgUp/PgDn	orbit, zoom, higher / lower	`1–7`	each lamp on / off
`W A S D, Q E`	walk; fly down / up	`-, =, \`	ambient / diffuse / specular
Left click	open or close a door	`P`	lighting presets
`Tab, I J K L U N`	select; move an object	`G, Y, Z, R`	Phong / Gouraud, soft shadows, refraction, ray tracing
`[ ] ; ' , .`	rotate, tilt, scale it	`H, O, F, X`	roof, outlines, fog, steam
`Space, Esc`	pause, quit	`Enter, M`	start the game, back to exploring

Table 9: The keyboard and mouse controls.

3.12 The ramen cooking game

A thin game layer on top of the shop (`game.cpp`); it is opt-in, so the program starts in explore mode and `Enter` opens it. A state machine drives it (Figure 13): a customer walks in and sits down and an order appears; the player takes raw noodles (1) to the noodle pot, takes a bowl (2), scoops the noodles when they look perfectly boiled, ladles broth, adds toppings (3–6) at the prep area, and serves the bowl. The noodle colour changes with the boiling time, and the result is graded:

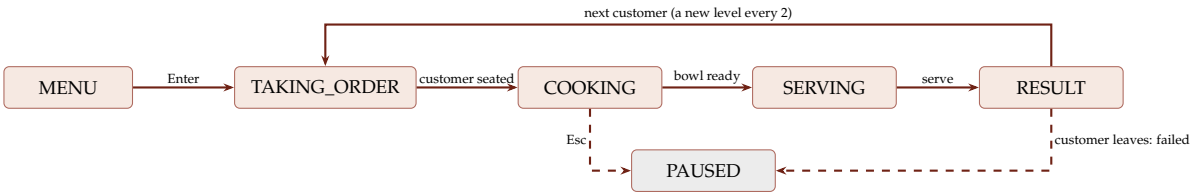
$$Q = 0.5\,\text{accuracy} + 0.35\,\text{cook} + 0.15\,\text{time}, \qquad \text{stars} = \text{round}(5Q), \tag{8}$$

where accuracy is the matched share of required items minus extras, *cook* is 0.15, 0.60, 1.00, 0.60, 0.10 for raw (< 3 s), undercooked, perfect (6–9 s), overcooked and ruined (> 12 s), and *time* is the patience left (from $105 - 14(\text{level} - 1)$ s, at least 50 s). Money is $\lfloor 10\,Q\,(1 + 0.1(\text{level} - 1)) + 0.5\,\text{stars} \rfloor \times 10$ yen, plus 20 yen for a five-star bowl; a new level (up to 4) starts every two customers and adds toppings and harder orders.

4 Results

4.1 Outdoors

Figure 14 shows the shop by day and at night. By day the sun and sky light fill the scene and trees cast shadows on the lawn, sidewalk and street; at night the street lamps and lanterns glow, the shop windows shine through the paper, and fireflies drift over the grass. The street is alive with the cyclist, the person and the cat, and the pond behind the shop has two ducks swimming around the lily pads.





(a) Title screen **(b)** Order board and prompts **(c)** A customer at the counter

Figure 13: The game: its state machine and three screens (HUD drawn with an orthographic projection).



(a) Day **(b)** Night **(c)** Pond view with ducks

Figure 14: The shop from outside and the lake behind it.

4.2 Ground floor

Inside, the counter, kitchen, tables and lamps show most of the lighting and material work (Figure 15): the lacquered counter top and the steel pots catch bright highlights, the pendant and paper lanterns warm the wood, the table lamps (bell-shaped glass-and-wood pendants) cast cones of light onto the tables, and the stools throw soft shadows.



(a) Counter and stools **(b)** Kitchen and counter top **(c)** Customer table

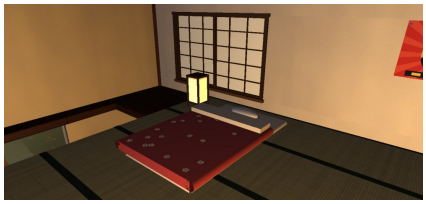
Figure 15: The ground floor.

4.3 Upstairs tatami room and details

The upper room (Figure 16) is a cosy Japanese bedroom: woven tatami, plaster walls with a timber frame, fusuma doors with a checker band, a board ceiling with a square paper lamp, a futon with a flower quilt, a tea set on the chabudai, framed pictures drawn in code (Mt. Fuji, lucky cat, onigiri, koi, a great wave and a ramen poster), windows that let the daylight in, and two fans. The wind chime under the porch awning and the nobori banner by the door complete the front.


(a) Room view


(b) Windows, lamp and fan


(c) Futon and andon lamp


(d) Wall pictures


(e) Porch with banner and chime

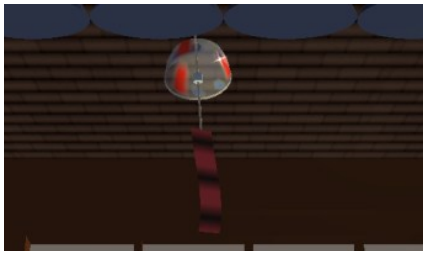

(f) Wind chime (cropped)

Figure 16: The upstairs room, the porch details and the wind chime.

4.4 Summary and performance

Item	Value	Item	Value
Lights / switchable fixtures	8 / 7	Camera modes	7
Procedural textures	13	Shadow map	512×512, PCF 3×3
Ray-traced objects	14	Ray-trace cost (Release / Debug)	about 13 ms / 80 ms
Cooking-game stations	4	Noodle states, levels	5, up to 4
Source files, lines of code	35, about 15,000	External assets	none

Table 10: Scene statistics. The ray-trace time was measured in a close view with 6–7 featured objects (about 7,000 eye rays) on 8 threads.

Parameter	File	What changing it does
`SHADOW_MAP_SIZE` (512)	`shadowmap.cpp`	1024 gives sharper soft shadows at a higher GPU cost
Spot cut-off 28°, exponent 35	`lighting.cpp`	wider cone / softer centre of the table lamps
k_l, k_q of each light	`lighting.cpp`	how far each lamp reaches (pool of light)
`TARGET_MS` (25)	`rtpass.cpp`	smaller: smoother frame rate but blockier ray tracing
`TREE_LOD_DIST` (18)	`exterior.cpp`	distance at which trees switch to the lighter build
Wind-chime gains k_1, k_2	`scene.cpp`	how strongly the wind pushes the bell and the strip
Cyclist, person and cat speeds	`scene.cpp`	how fast they move along the street

Table 11: Parameters that can be changed to see their effect.

5 Discussion

What worked. Building every object from a few primitives kept the code small and the scene consistent, and the per-light switches made the lighting model easy to inspect. The shadow stencil trick gives clean, non-overlapping tree shadows cheaply, and the hybrid of rasterisation with a small ray tracer shows real reflection and refraction without tracing the whole scene.

Problems and how they were solved.

- *Only 8 lights.* The outdoor lights are re-assigned to interior fixtures while the inside is drawn, and a shader mask keeps outside lights from lighting the interior through the walls.
- *Lights switched off but the room still lit.* A global ambient and glowing materials caused it; the global ambient is now zero and glowing surfaces fade with their lamp, so every term really belongs to a light.
- *Broken, streaky shadows.* The shadow planes were a few millimetres above the ground and z-fought, and leaf cards left gaps in the stencil; depth testing is off for that pass, and trees use full leaf geometry near the shop and solid crowns far away.
- *Blocky ray-traced pots up close.* The ray budget keeps the cost bounded, so a large on-screen pot gets one ray per several pixels; it is crisp at normal distances and in Release builds.

Limitations. Refraction through glass is a screen-space approximation; ray tracing covers only the featured interior objects and not the outdoor scene; with full leaf geometry the shadow edge is slightly broken where

light passes between leaf cards; the night shadows outdoors are simple fixed shapes; and there is no sound. The scene is designed for a desktop GPU with OpenGL 3.0.

Possible improvements. Cube-map shadow maps for each lamp, a shadow map for the sun, ray tracing in a compute shader with a GPU BVH, and customers that walk into the shop by themselves.

6 Conclusion

The Ramen Shop is a complete, explorable 3D scene written from scratch with OpenGL. It shows modelling from basic shapes, the full set of 3D transformations in hierarchical models, procedural textures, the ambient, diffuse and specular illumination model with eight individually switchable lights, Gouraud and Phong shading, three kinds of shadows, screen-space glass refraction, real ray tracing with a BVH, rich animation (including a physically simulated wind chime) and a playable cooking game. All of it runs interactively from a single Visual Studio project with no external assets.

References

[1] B. T. Phong, *Illumination for computer generated pictures*, CACM 18(6), 1975.

[2] J. F. Blinn, *Models of light reflection for computer synthesized pictures*, SIGGRAPH 1977.

[3] T. Whitted, *An improved illumination model for shaded display*, CACM 23(6), 1980.

[4] L. Williams, *Casting curved shadows on curved surfaces*, SIGGRAPH 1978; W. Reeves et al., *Rendering antialiased shadows with depth maps*, SIGGRAPH 1987.

[5] C. Schlick, *An inexpensive BRDF model for physically-based rendering*, Eurographics 1994.

[6] D. Shreiner et al., *OpenGL Programming Guide*, Addison-Wesley.